AI RECEPTIONIST

Self-Hosted Voice Agent · Pipecat + FastAPI + Next.js Monorepo

TECHNICAL IMPLEMENTATION SCOPE

Version 1.0 · ai-receptionist v0.1.0 · 2026 · Confidential & Proprietary

65 files	4 services	~2.5K LOC	6 weeks
Scaffold	Monorepo	To Implement	Build ETA

01 Executive Summary

Engineering build plan for the AI Receptionist scaffold

The AI Receptionist repository is currently in a Scaffold state — topology, services, configuration, and skeleton code are in place, but most modules are stubs awaiting real implementation, API keys, and tuning. This document is the companion to the Full Scope and exists for engineers: it enumerates every TODO, defines the exact build order, specifies the API contracts, and prescribes the test strategy that lets us ship the MVP without regressing prompt behaviour.

Dimension	Detail
Repo	C:\Users\Admin\Desktop\PROJECTS\NEW\Receptionist (monorepo, 4 services)
Current State	Scaffold — topology, config, skeleton code in place; modules stubbed
Target	Phases 0–3 shippable (24/7 inbound, take-message, booking, dashboard v1)
Stack (frozen)	Pipecat · FastAPI · Postgres+pgvector · Redis+arq · Next.js 14
Build Owner	1 backend + 1 agent + 1 frontend (3 engineers, ~6 weeks)
Definition of Done	All Phase-0–3 acceptance criteria in §17 met; baseline eval passes

How to read this document

Sections 02–03 set the order of operations. Section 04 is the schema-level truth. Sections 05–09 define contracts the agent and api must satisfy. Sections 10–13 define what "tested" means. Sections 14–16 are ops. Section 17 is the exit criteria for each phase of the Full Scope.

02 Build Order & Dependencies

Module-by-module sequence; parallelisable tracks marked

The build follows the dependency graph of the call lifecycle: nothing useful happens until a call can ring, transcribe, respond, and persist. We layer in this order; each track is owned by one engineer and tracked independently.

PHASE 1 — Foundation & Wiring — Days 1–4

Env & secrets

- Fill .env from .env.example — required keys only (POSTGRES_PASSWORD, SECRET_KEY, NEXTAUTH_SECRET, provider keys)
- docker compose up — verify all containers healthy
- make migrate — confirm pgvector extension loaded
- make seed — confirm first business + admin user created

Service contracts

- Freeze api/main.py route surface (no new routes without doc update)
- Freeze internal router shape — agent ↔ api only ever writes via /internal/*
- Lock idempotency-key header convention (Idempotency-Key:) for all mutations

PHASE 2 — Telephony & Pipeline Skeleton — Days 5–8

Telnyx inbound

- api/routers/internal.py + webhook signature verification (Ed25519)
- agent/providers/telnyx.py serializer — 8 kHz μ-law frames
- agent/pipeline.py — assemble empty Pipecat pipeline; call connects & hangs up cleanly

Cost & turns ledger

- agent/cost.py — running per-turn ticker (LLM tokens, STT seconds, TTS chars)
- api/routers/internal.py — POST /internal/call-turns (agent → api persistence)
- api/routers/internal.py — POST /internal/calls/{id}/finalize (cost rollup)

PHASE 3 — Conversation Core — Days 9–14

STT + LLM + TTS

- Wire Deepgram streaming → Claude Haiku 4.5 → Cartesia Sonic-2
- agent/fallbacks.py — primary→Gemini fallback on timeout / 5xx
- agent/prompts/builder.py — assemble prompt from prompt_versions row + dynamic suffix

P0 tools

- tools/messages.py (take_message) — server-validated, idempotent
- tools/transfer.py (transfer_call) — Telnyx call-control REST call
- tools/transfer.py (end_call) — graceful hangup

Latency instrumentation

- Per-turn latency_ms capture at 4 stages (endpoint, STT, LLM-ttfb, TTS-ttfb)
- Emit histogram metrics to Prometheus

PHASE 4 — Booking + RAG — Days 15–20

Calendar

- tools/calendar.py — Google Calendar FreeBusy + events.insert
- Idempotency key on (caller_phone, slot_start)
- Re-check availability at booking time (race protection)

Knowledge base

- tools/knowledge.py — pgvector cosine similarity search
- tools/knowledge.py — log_unanswered writes to knowledge_gaps
- Seed 20+ FAQs from the prompt-validation corpus

SMS confirmations

- tools/sms.py — template-rendered, server-side only (no raw model strings)

PHASE 5 — Post-call & Notifications — Days 21–24

Post-call job

- api/jobs/post_call.py — load transcript, LLM summary, classify intent + outcome + sentiment
- api/jobs/post_call.py — compute final cost from call_turns + telephony minutes
- api/jobs/post_call.py — enqueue notifications (email/SMS/Telegram/Slack)

Daily report

- api/jobs/daily_report.py — aggregate calls, costs, escalations; send via configured channels

PHASE 6 — Dashboard v1 — Days 25–32

Read paths

- /calls list + /calls/[id] synced audio + transcript view
- /messages inbox + status transitions
- /appointments list + cancel action
- /config prompt-version viewer (read-only)

Write paths

- POST /config/prompt-versions (save draft) + publish action

- POST /knowledge (add FAQ) + edit + delete
- POST /contacts/{id}/block + unblock
- POST /calls/{id}/flag

PHASE 7 — Hardening & Launch — Days 33–42

Eval harness

- eval/run_eval.py — full scenario runner with assertions
- make eval-baseline — capture Phase-2 behaviour as regression baseline

Observability

- Sentry wiring (api + agent)
- Langfuse prompt-eval integration
- Prometheus scrape config + Grafana dashboard JSON

Launch readiness

- Caddy TLS auto-renew verified
- Backups: nightly pg_dump → S3
- Soft launch on after-hours only; promote to full-hours after 1 week stable

03 Stub-Completion Checklist

Every TODO in the scaffold, mapped to the module that closes it

These are the work items already enumerated in the Full Scope §17, re-keyed here by file path with the closure criteria. Treat each as a single PR — reviewers should be able to verify the closure criterion without reading the surrounding module.

File	What is stubbed	Closure criterion
agent/pipeline.py	Caller lookup + turn persistence hooks	Caller fetched from /internal/contacts/lookup
agent/tools/transfer.py	Real Telnyx call-control call	POST https://api.telnyx.com/v2/calls/{id}/ac
agent/tools/calendar.py	Google Calendar FreeBusy + events.insert	OAuth refresh-token flow wired; FreeBusy c
api/jobs/post_call.py	Transcript load, summary LLM call, notifications queue	Persists summary + intent/outcome/sentime
eval/run_eval.py	Scenario execution + assertions	Runs each YAML scenario against the agen
api/routers/internal.py	Agent → api write endpoints	POST /internal/call-turns (auth: INTERNAL_
api/worker.py	arq worker wiring	Boots arq with RedisSettings; registers pos
dashboard/app/config/page.tsx	Prompt-version create / publish UI	Form + draft save + version diff + publish b
dashboard/app/knowledge/page.tsx	Knowledge CRUD	Add FAQ form, edit modal, delete confirmat
dashboard/app/contacts/page.tsx	Block / unblock action	Confirm modal; mutation invalidates /contac
dashboard/app/appointments/page.tsx	Cancel appointment UI	Cancel mutation + optimistic update; cance
dashboard/app/calls/[id]/page.tsx	Flag + flag_note UI	Flag toggle + reason input; mutation logged
dashboard/app/settings/page.tsx	Settings forms (channels, retention, users)	Forms per section; saves go through PATC
dashboard/app/quality/page.tsx	Review queue	List flagged calls + LLM-judge regression re

04 Data Model — Deltas & Indexes

Schema changes required beyond migration 0001_initial

Schema additions (migration 0002)

Table	Column / Constraint	Why
calls	partial index on flagged WHERE flagged = true	Dashboard review queue + quality page filter
calls	composite index (business_id, started_at DESC)	Overview + calls list page are ordered + tena
messages	composite index (status, created_at DESC)	Inbox sort by status + recency
appointments	composite index (business_id, starts_at)	Day-view in dashboard; booking-window vali
call_turns	composite index (call_id, seq)	Transcript replay in /calls/[id] is the hot path
knowledge_items	HNSW index on embedding vector (cosine, m=16, ef_construction=64)	Self-serve search at 10K rows; tune
prompt_versions	partial unique index WHERE status = live	Enforce exactly one live prompt per business
audit_log	composite index (business_id, created_at DESC)	Settings → audit view is always tenant-scope
contacts	partial index WHERE blocked = true	Caller lookup short-circuits blocked contacts

Field-level notes

- calls.cost_telephony / cost_stt / cost_llm / cost_tts are Numeric(10,5) — never float; sums computed in DB.
- call_turns.tool_args / tool_result are JSONB; validate against tool schema on insert (defense vs prompt injection).
- prompt_versions.blob JSONB keys: persona, style, business_facts, policies, actions, rules, escalation — exactly these seven, no more.
- knowledge_items.embedding is vector(768) — locked to the OpenAI text-embedding-3-small dimension in Phase 2 (do not swap mid-flight).
- appointments.external_event_id carries the Calendar event ID; nullable until Phase 3 sync is on.
- users has bcrypt hash + jose-signed HS256 JWT only — no OAuth until 2FA lands in Phase 4.

Retention policy implementation

Table	Default	Configurable via	Job
calls, call_turns	Indefinite	settings.retention_days_calls	cron: DELETE WHERE started_at < n
recordings (object storage)	30 days	settings.retention_days_recordings	cron: list + delete orphan keys in REC
audit_log	Indefinite	—	— (regulatory: keep all config changes

05 API Contracts

All HTTP endpoints; auth model; idempotency; pagination; errors

Authentication

- Dashboard → api: Bearer HS256 JWT (jose), 15-min access + 7-day refresh cookie (HttpOnly, Secure, SameSite=Lax).
- Agent → api (internal): static X-Internal-Token header; token only valid on the docker network — Caddy refuses to forward /internal/* externally.
- Telnyx → api/webhooks: Ed25519 signature verified against TELNYX_PUBLIC_KEY over the raw request bytes.
- Tool server-side validation: every value the LLM emits is re-validated server-side (zod) before reaching a provider.

Dashboard API surface

Method	Path	Auth	Notes
POST	/auth/login	none	bcrypt verify → JWT pair; sets refresh cookie
POST	/auth/refresh	refresh cookie	Rotates access token; refresh cookie unchanged
GET	/calls	JWT	Query: from, to, flagged, intent; paginated cursor
GET	/calls/{id}	JWT	Includes turn list + recording_url + cost breakdown
POST	/calls/{id}/flag	JWT	Body: { flag_note } — writes audit_log
GET	/messages	JWT	Filter by status; cursor pagination
PATCH	/messages/{id}	JWT	Status: in_progress / done
GET	/appointments	JWT	Filter by date range + status
PATCH	/appointments/{id}	JWT	cancel → also cancels Calendar event
GET	/contacts	JWT	Filter: blocked, tag
POST	/contacts/{id}/block	JWT	Idempotent; logs to audit_log
GET	/config/prompt-versions	JWT	List all versions + status
POST	/config/prompt-versions	JWT	Create draft; status=draft
POST	/config/prompt-versions/{id}/publish	JWT	Sets to live; archives previous live row (DB partial unique enf
GET	/knowledge	JWT	List FAQ items
POST	/knowledge	JWT	Body: question, answer → server embeds & stores
DELETE	/knowledge/{id}	JWT	Soft-delete (status=archived)

Internal router (agent → api)

Method	Path	Body	Notes
POST	/internal/contacts/lookup	{ phone_e164 }	Returns { contact_id, blocked, la
POST	/internal/calls	{ call_sid, from, to, business_id }	Idempotent on call_sid — UPSE
POST	/internal/call-turns	{ call_id, role, text?, tool_name?, tool_args?, tool_result?, latency_ms? }	Appends one turn;
POST	/internal/calls/{id}/finalize	{ ended_at, duration_sec }	Triggers post_call job enqueue;
GET	/internal/calls/{id}/prompt	—	Returns current live prompt blob
GET	/internal/knowledge/search	?q=&k=	Returns top-k snippets with simil
POST	/internal/knowledge/gaps	{ call_id, question }	Writes knowledge_gaps row
POST	/internal/messages	{ call_id?, caller_phone, name?, reason, callback_window? }	Inserts message; dedupe on (caller_phone, rea

Error model

Status	When	Body shape
400	zod validation failure on body	{ error: "validation", issues: [{path, message}] }
401	Missing or invalid JWT / Telnyx sig / internal token	{ error: "unauthorized" }
403	JWT valid but business_id mismatch	{ error: "forbidden" }
404	Resource not found within tenant scope	{ error: "not_found" }
409	Idempotency conflict (same key, different body)	{ error: "idempotency_conflict" }
422	Provider rejected (Calendar slot taken, Telnyx 4xx)	{ error: "provider", provider: "...", code: "..." }
429	Per-tenant or per-number rate limit	{ error: "rate_limited", retry_after_ms }
500	Unexpected — logged with trace_id	{ error: "internal", trace_id }

06 Agent Pipeline — Frame Flow & Persistence

Pipecat pipeline assembly, frame instrumentation, persistence hooks

The agent pipeline is a serial Pipecat pipeline with three side effects per turn: latency capture, cost accrual, and persistence. Below is the frame flow for one caller turn → agent response cycle.

Stage	Service	Frame out	Side effect
1	Telnyx transport	AudioFrame (8 kHz μ-law)	—
2	Silero VAD (Pipecat turn detection)	UserStartedSpeakingFrame / UserStoppedSpeakingFrame	Start STT latency timer (T0)
3	Deepgram STT (streaming)	TranscriptionFrame (partials + final)	On final: stop STT timer → T_stt = T1-T0
4	Context service	MessagesFrame (system + history + new turn)	Resolve live prompt blob via /internal/calls/{id}/p
5	Claude Haiku 4.5 (with_fallback → Gemini 2.5 Flash-Lite)	LLMTextFrame + ToolUseFrame	Stop LLM-ttfb timer → T_llm = T2-T1; accumula
6	Tool runner (one or more in sequence)	ToolResultFrame	For each tool: 2-3s timeout; POST /internal/* on
7	Cartesia Sonic-2 TTS	TTSAudioFrame	Stop TTS-ttfb timer → T_tts = T3-T2; accumulat
8	Telnyx transport (egress)	AudioFrame	—
9	Turn persistence	POST /internal/call-turns (x1 per role + tool)	Async, batched; seq monotonic

Latency budget enforcement

- Per-turn total budget: 800 ms (T_stt + T_llm + T_tts).
- Hard timeout per LLM call: 4 s — past that, with_fallback flips to Gemini.
- Tool timeout: 2 s read-only (calendar, knowledge), 3 s write (booking, transfer).
- If total exceeds 5 s end-to-end → emit metric voice_agent_turn_overrun_total{stage="total"}.
- Dashboard analytics page renders p50 / p95 of T_stt, T_llm, T_tts (Prometheus histograms).

Persistence hook placement

- On call_started → POST /internal/calls (idempotent on call_sid).
- On every LLM TextFrame → POST /internal/call-turns (role=agent).
- On every STT final → POST /internal/call-turns (role=caller).
- On every ToolUseFrame / ToolResultFrame → POST /internal/call-turns (role=tool, tool_args + tool_result JSONB).
- On call_ended → POST /internal/calls/{id}/finalize → enqueue post_call job.
- Per-turn persistence is async — buffered, flushed every 250 ms or 20 rows, whichever.

07 Tool Implementation Specs

Per-tool contract, server-side validation, idempotency, timeouts

Every tool validates its arguments server-side (the model is untrusted). Idempotency keys are required on every mutation. Timeouts are short. Below is the canonical shape; each tools/*.py file implements exactly one row.

take_message

P0 • read+write

- Args (zod): { name: string max 80, phone: string E.164, reason: string max 500, callback_window?: enum[today, evening, tomorrow, anytime] }
- Server-side: phone normalised to E.164; reason length checked; existing open message for same (phone, reason) → 200 with existing id (idempotent).
- Side effect: POST /internal/messages; notify configured channels (email default).
- Timeout: 2 s; on 5xx → model instructed to retry once then end_call.

transfer_call

P0 • write

- Args (zod): { department: enum[sales, support, billing, manager] }
- Server: POST https://api.telnyx.com/v2/calls/{call_control_id}/actions/transfer with target number from settings.transfer_numbers[department].
- On Telnyx 4xx (no answer, busy) → fall back to take_message; model told to apologise and capture offer.
- Timeout: 3 s; logged to audit_log on success.

end_call

P0 • write

- Args (zod): { reason: enum[completed, transferred, failed, timeout] }
- Server: POST https://api.telnyx.com/v2/calls/{id}/actions/hangup.
- No body to model — required by end-of-turn logic.
- Triggers /internal/calls/{id}/finalize.

check_availability

P1 • read

- Args (zod): { date: ISO date, service?: string, tz?: IANA }
- Server: Google Calendar FreeBusy on settings.default_calendar_id; 30s Redis cache.
- Returns: array of { starts_at, ends_at } open slots, max 10.
- Timeout: 2 s.

book_appointment

P1 • write

- Args (zod): { starts_at: ISO datetime, service: string, name, phone, notes? }
- Idempotency key: hash(caller_phone + starts_at + service). Re-running with same key returns existing booking.
- Race protection: re-check availability at booking time before events.insert.
- Server: events.insert on default calendar → store external_event_id; status=booked.
- Timeout: 3 s; on conflict (slot taken) → return 422 with alternative slots.

send_sms

P1 • write

- Args (zod): { template: enum[booking_link, scan_intro, callback_promised], variables: object }
- Server: looks up template body from settings.sms_templates[template]; renders variables server-side; POST Telnyx messaging.
- Model never produces raw SMS body — defeats prompt injection for outbound messages.
- Timeout: 2 s; logs sender + recipient + template_id to audit_log.

search_knowledge

P1 • read

- Args (zod): { question: string max 200, k?: int default 3 }
- Server: embed via text-embedding-3-small; pgvector cosine similarity search; return top-k with score.
- Threshold: drop results below 0.72 similarity.
- Timeout: 1.5 s.

log_unanswered

P1 • write

- Args (zod): { question: string max 200 }
- Server: POST /internal/knowledge/gaps — staff resolves later by adding an FAQ.
- No model-visible side effect; never blocks the turn.

08 Webhook Verification

Telnyx Ed25519 signature flow on raw request bytes

Flow

- Telnyx POSTs to /webhooks/telnyx with header telnyx-signature-ed25519:
- The signature is over the raw request body — must be captured before any JSON parsing.
- Verification: pynacl / nacl.signing.VerifyKey(TELNYX_PUBLIC_KEY).verify(body, signature_b64).
- On success → enqueue internal action (open media stream, hangup, etc.).
- On failure → HTTP 401 + log with trace_id; emit metric webhook_signature_fail_total.
- Replay protection: enforce timestamp window (default 5 min) — reject signatures with timestamp older than now-5min.

Implementation notes

- Use FastAPI dependency get_raw_body(request: Request) that returns await request.body() — this must be the first byte buffer read.
- Verify BEFORE pydantic validation — never let invalid bodies advance into the queue.
- Caddy must NOT buffer the body for /webhooks/telnyx — set flush_interval -1 or use request_body buffer 0.
- In test: provide a fixture that signs with the test private key (TELNYX_PUBLIC_KEY overridden in conftest.py).

09 Cost Tracker — Per-Turn Attribution

agent/cost.py algorithm; rounding rules; persistence shape

Pricing table (loaded from settings, mutable per business)

Component	Unit	Default USD/unit	Source
Telephony (Telnyx inbound)	minute	0.0085	settings.cost_telephony_per_min
STT (Deepgram Nova-3)	second	0.0000717	settings.cost_stt_per_sec — ~$0.0043
TTS (Cartesia Sonic-2)	character	0.000015	settings.cost_tts_per_char — ~$0.015
LLM primary (Haiku 4.5)	1K tokens	input 0.001 / output 0.005	settings.cost_llm_in_per_1k + cost_llm
LLM fallback (Gemini 2.5 Flash-Lite)	1K tokens	input 0.000075 / output 0.0003	settings.cost_llm_fallback_in_per_1k +

Per-turn computation

Each turn records usage at the moment of occurrence. The CostTracker instance is keyed by call_id and held in the per-call context; on turn end the running totals are POSTed to /internal/call-turns alongside the turn row.

```
class CostTracker:
■ def on_stt_final(self, audio_secs): self.cost_stt += audio_secs * STT_PER_SEC
■ def on_llm_response(self, in_tok, out_tok, fallback=False): self.cost_llm += in_tok *
IN_PER_1K/1000 + out_tok * OUT_PER_1K/1000
■ def on_tts_audio(self, chars): self.cost_tts += chars * TTS_PER_CHAR
■ def on_call_end(self, duration_sec): self.cost_telephony += (duration_sec/60) * TEL_PER_MIN
■ def snapshot(self) -> CallCost: round(Numeric, 5 places) per field
■ def flush(self): POST /internal/call-turns x N rows (one per usage event)
```

On-call finalize

- On call_ended → POST /internal/calls/{id}/finalize → api computes final cost_telephony from duration_sec, sums cost_*** from call_turns rows.
- Numeric(10,5) stored — sum computed in DB (never in Python) to avoid float drift.
- Dashboard cost-per-call metric = SUM(cost_*) / COUNT(calls) over date range.

10 Post-Call Job Specification

api/jobs/post_call.py — summary, classification, notification dispatch

Job trigger

- Trigger: POST /internal/calls/{id}/finalize → api/jobs/post_call.py enqueued via arq.
- Retries: 3 attempts with exponential backoff (1s, 4s, 16s); on give-up → alert + dead-letter queue.
- Idempotent: re-running with same call_id re-uses existing summary (UPSERT pattern).

Job steps

#	Step	Source / API	Timeout
1	Load call_turns ordered by seq	DB query (indexed on (call_id, seq))	500 ms
2	Render transcript (caller + agent text frames only)	in-memory	100 ms
3	LLM summary call — 2-3 sentences	Claude Haiku 4.5, max_tokens=120, temp=0.2	4 s
4	Classify intent (enum) + outcome (enum) + sentiment (enum)	Claude Haiku 4.5 structured output (JSON schema)	4 s
5	Compute telephony cost from duration_sec	duration_sec * cost_telephony_per_min	—
6	Sum STT/LLM/TTS costs from call_turns rows	SQL aggregate (avoid Python sum on floats)	100 ms
7	UPSERT calls row (summary, intent, outcome, sentiment, cost_…)	INSERT … ON CONFLICT (id) DO UPDATE	200 ms
8	Upload recording (if Telnyc recording enabled)	S3-compatible PUT to RECORDINGS_BUCKET	5 s
9	Enqueue notifications	arq job notifications.notify_call_completed	—

Notification dispatch (api/jobs/notifications.py)

- Always: email to business owner (SMTP / Resend / SendGrid — first configured wins).
- Optional: Telegram, Slack webhook — fired in parallel; one failure does not block the others.
- Payload: call summary, recording URL, transcript link, callback info if take_message triggered.
- Rate limit: max 5 notifications per call (deduplicates by template + call_id).
- Quiet hours: respect business settings.quiet_hours — defer non-urgent pings (e.g. Telegram) outside the window.

11 Prompt Caching Strategy

Section boundaries, cache hit expectations, iteration workflow

Static prefix (cacheable across calls — same business_id)

Position	Section	Tokens	Cache key
1	persona	~30	business_id + version_id
2	style	~150	business_id + version_id
3	business_facts	300–1000	business_id + version_id
4	policies	~200	business_id + version_id
5	actions	~150	business_id + version_id
6	rules	~150	business_id + version_id
7	escalation	~100	business_id + version_id
subtotal	Static prefix	~1,100–1,800 tokens	Anthropic cache (TTL 5 min, refreshed per

Dynamic suffix (per call)

- now() formatted in business tz
- caller_phone (last 4 only — never full)
- caller history: tags, last_call_at, total_calls (no transcript text)
- open message count, next appointment time
- system reminder: "today is {weekday}, business hours {open}–{close}; if outside hours, say so first"
- Section ordering is fixed — never insert dynamic content above the rules section (would break cache).

Iteration discipline

- Edit in dashboard → Save as draft → status=draft; live row untouched.
- Run make eval against draft (--prompt-version=ID).
- When metrics ≥ baseline → Publish; previous live row → archived (partial unique index ensures ≤1 live).
- Roll back by re-publishing an archived version.
- Never hot-edit the live prompt — always through version → publish flow.
- cache_control markers placed exactly at the start of the static prefix and after section 7 — Claude caches from marker to marker.

12 Eval Harness Implementation

eval/run_eval.py execution model, assertions, baseline regression

Execution model

- eval/run_eval.py runs each YAML scenario against agent service with --suite eval/scenarios.
- Tools are stubbed (mock implementations injected via fixture); only LLM is real.
- For each caller turn → capture agent response + any tool calls → apply expectations → mark pass/fail.
- No audio — text-mode only (keeps eval fast and CI-cheap).
- Output: results.json — per scenario { pass, fail, assertions: [{name, passed, expected, actual}] }.

Expectation schema

```
turns:
  - role: caller
    text: "I'd like to book a haircut tomorrow at 2pm."
    expect:
      - agent_uses_tool: check_availability
      - agent_eventually_uses_tool: book_appointment
      - agent_response_length_max_words: 25
      - agent_offer_take_message: false

metrics:
  - tool_call_accuracy # fraction of turns where expected tool fires
  - brevity # p50 turn word count
  - no_hallucinated_facts # all facts trace to knowledge_items or business_facts
```

Baseline & regression

- make eval-baseline — runs eval, writes results.json → eval/baselines/<name>.json.
- CI on PR: run eval, compare to baseline; fail if tool_call_accuracy regresses > 5% or brevity p50 > 30 words.
- Phase 4: wire to Langfuse — track per-scenario traces + judge model for open-ended assertions.
- TODO: prompt-injection scenarios, language-switch scenarios.

13 Test Strategy

Unit / integration / e2e / load matrix; what to mock; coverage targets

Test pyramid

Layer	Tool	What it covers	Coverage target
Unit	pytest	pure logic: cost math, idempotency, validation, prompt assembly	80% of app + agent modules
Integration	pytest + testcontainers	api ↔ Postgres (pgvector) ↔ Redis; tool calls against routers, adapters	all routers, all tools, all provider resp
Contract	schemathesis (or hand-rolled)	OpenAPI schema validates every endpoint; 4xx/5xx response shapes	every endpoint, every documented
Eval	pytest + YAML scenarios	prompt behaviour regression (see §12)	all 6 baseline scenarios + 2
e2e	docker compose up + synthetic Telnyx webhooks	call.initiated → answered → tool → hangup → post_call_job → email	1 happy path + 1 voicemail
Load	k6 (post-Phase-2)	20 concurrent calls sustained for 10 min; p95 turn latency	skeleton for Phase-2 capacity r

Mocking rules

- Always mock: Telnyx, Deepgram, Cartesia, Anthropic, Google Calendar, Gmail/SMTP.
- Never mock: Postgres, Redis (use testcontainers).
- For Pipecat pipeline tests — mock the transport services but run the real frame processor chain.
- LLM mock: a deterministic stub that returns canned tool calls + text for fixtures.
- Webhook signature tests: sign with a test private key in conftest; do not disable signature verification.

Critical tests (must exist before merge)

Test	Why critical
test_idempotency_messages.py — same Telnyx message_id twice → one row	Prevents duplicate writes on webhook retry
test_calendar_race.py — slot freed then re-booked within 200ms	Catches stale-availability check at booking time
test_webhook_signature.py — unsigned/bad-sig/missing-key all return 401	Security gate
test_prompt_caching.py — same prompt_version_id twice → cache_read_tokens > 0	Confirms Anthropic caching is actually engaged
test_cost_rollup.py — sum across 10 turns matches call_log.cost_usd	Catches drift between per-turn and per-call accounting
test_call_duration_cap.py — calls > MAX_CALL_DURATION_SEC are force-ended	Cost control; prevents runaway LLM loops

14 Observability Wiring

Metric names, structured log keys, trace spans, dashboards

Prometheus metrics (api + agent)

Metric	Type	Labels	Used by
voice_agent_turn_latency_seconds	histogram	stage=endpoint|stt|llm|tts|total	p50/p95 latency dashboa
voice_agent_turn_overrun_total	counter	stage	alert: overrun rate > 10%
voice_agent_cost_dollars_total	counter	component=telephony|stt|llm|tts, business_id	cost-per-call dashboard
voice_agent_tool_calls_total	counter	tool, result=success|fail|timeout	tool reliability + per-tool la
api_http_requests_total	counter	method, path, status	request rate + error rate
api_http_request_duration_seconds	histogram	method, path	api p95 latency
api_jobs_processed_total	counter	job=post_call|notifications|daily_report, result	job queue health
webhook_signature_fail_total	counter	—	security alert: > 0 sustain
llm_fallback_engaged_total	counter	from=primary|fallback	alert: > 5% of turns mean

Structured log keys (structlog)

Key	Type	Example
call_id	uuid	8b3e... (every log line within a call shares this)
business_id	uuid	tenant scope
caller_phone_hash	sha256(last 4)	phone_e164 — never full ring
turn_seq	int	monotonic per call
latency_ms	object	{ endpoint, stt, llm, tts, total }
tool_name	string|null	check_availability / book_appointment / …
tool_result	enum	success | validation_error | timeout | provider_error
cost_usd	object	{ telephony, stt, llm, tts }
prompt_version_id	uuid	correlation behaviour ↔ prompt change

Trace spans (api → agent → providers)

- api.receive.webhook → agent.call.connect → agent.turn.{stt,llm,tool,tts} → agent.call.end
- api.jobs.post_call — children: api.post_call.summary, api.post_call.classify, api.post_call.notify
- Tool spans — one per tool call, includes tool_name, duration, status
- Export to Sentry; for Phase 4 wire to OpenTelemetry collector + Langfuse for LLM traces.

15 Build & Deploy Pipeline

Image build order, migration runbook, rollback procedure

Image build

Service	Base image	Build context	Cache hint
api	python:3.11-slim	pyproject.toml + api/** + shared/**	uv sync --frozen --all-groups as cache key
agent	python:3.11-slim	pyproject.toml + agent/** + shared/**	share uv cache layer with api
worker	python:3.11-slim	pyproject.toml + api/**	same base as api — same Dockerfile, diffe
dashboard	node:20-alpine	dashboard/**	npm ci with package-lock.json cached

Migration runbook

- make migrate — runs Alembic upgrade head inside the api container.
- make migrate-new m="..." — generates new revision; review for downgrade.
- Backward-compat rules: never drop a column in the same release that removes the reader; two-release deprecation.
- Before deploy: docker compose run --rm api alembic current → capture for /persist
- On failed migration: docker compose run --rm api alembic downgrade -1; investigate; do not force.
- Long migrations (>30s on prod data): pre-deploy script that runs against a shadow table first.

Rollback procedure

- git pull && git checkout
- docker compose build && docker compose up -d
- On schema incompatibility: alembic downgrade -1 (only safe for additive migrations).
- Call forwarding fallback: Telnyx fallback number auto-routes to a human line if api health probe is down.
- Active calls during deploy: agent drains gracefully (signal handler — finishes current turn, then closes).
- Roll-forward preferred over roll-back once data has been written by the new code.

16 Open Implementation Risks

Provider behaviour, capacity assumptions, race conditions, TBDs

Risk	Likelihood	Impact	Mitigation
Pipecat version churn — API breaks between minor versions	Medium	High	Pin Pipecat to a known-good minor; review changelog; integration test
Telnyx call-control latency spike (> 800 ms)	Low	High	Pre-warm connection; fallback to take_message on timeout; alert on p9
Deepgram partials misaligned with turn detection	Medium	Medium	Always use Silero VAD — never silence-energy threshold; test with nor
Anthropic cache invalidation surprise (template change)	Low	Medium	cache_control markers pinned; alert on cache_hit_ratio drop
Google Calendar rate limits (FreeBusy = quota)	Medium	Medium	30s Redis cache on FreeBusy; exponential backoff; dashboard surface
Race: two parallel book_appointment calls for same slot	High	High	Idempotency on (caller_phone, slot) + re-check availability at booking t
arq worker crashes mid-job → partial state	Low	High	Idempotent UPSERTs; retries 3x; on give-up → dead-letter queue + ale
Cost drift: float arithmetic rolls up wrong by a fraction	Medium	Low	Numeric(10,5) in DB; sum in SQL; test_cost_rollup.py golden test
Dashboard XSS via transcript text (caller-controlled)	Low	High	React escapes by default; never use dangerouslySetInnerHTML; saniti
Provider key leaked via agent prompt (prompt injection)	Low	Critical	Agent has no secrets in context; every privileged action goes through a
Cartesia voice quota exceeded on spike day	Low	Medium	Pre-warmed connection; fallback voice id from settings; daily quota ale

17 Acceptance Criteria per Phase

Definition of done — each phase in the Full Scope is "done" when every box here ticks

Phase	Done when…
0 — Scope & Content	business hours/prices/FAQs in prompt_versions row; must/must-not list reviewed; voice pe
1 — Prompt validation	make eval passes on the locked prompt; baseline saved; tool_call_accuracy ≥ 90%; brevity
2 — Core voice agent (P0)	Telnyx number live; 24/7 inbound answers with AI disclosure; take_message / transfer_call
3 — Booking + dashboard v1	check_availability / book_appointment live with idempotency; post_call job produces summ
4 — Intelligence + polish	caller recognition + history; reschedule/cancel + SMS confirmations; knowledge base + RA
5 — Growth	multilingual coverage; CRM integration; outbound reminders; spam filtering + block list; qua

18 Conclusion & Next Steps

Summary and the first PR

This scope covers every TODO in the scaffold and the contracts that bind the four services together. The build order in §02 is the critical path; everything else is parallelisable. The acceptance criteria in §17 are the exit gates; nothing leaves a phase without all boxes ticked. The risks in §16 are the things we watch during the first two weeks of real traffic.

First PR (Day 1)

Step	Action	Owner
1	cp .env.example .env; fill required keys (POSTGRES_PASSWORD, SECRET_KEY, NEXTAUTH_SECRET, TELN	Backend lead
2	docker compose up — verify all 7 containers healthy (api, agent, worker, dashboard, caddy, postgres, redis)	Backend lead
3	make migrate; make seed — confirm first business + admin user	Backend lead
4	Wire POST /webhooks/telnyx + Ed25519 signature verification per SDK — unsigned requests with 401	Backend lead
5	Implement api/routers/internal.py: POST /internal/calls + POST /internal/calls/{id}/turns + POST /internal/calls/{id}/finalize	Agent lead
6	Assemble empty Pipecat pipeline in agent/pipeline.py — Telnyx in/out, call hangup cleanly	Agent lead
7	Add CostTracker stub in agent/cost.py — snapshot returns all zeros until wiring is exercised on every turn	Agent lead
8	CI: lint + pytest + make eval — must be green before any feature branch merges	All

AI Receptionist — a 24/7 voice receptionist that never sleeps, never misses a call, and never forgets a customer.

This document is confidential and proprietary. Not for distribution. Companion to the Full Scope — refer to that document for product, cost, and roadmap.